

The Shape of Stocks: Exploring the GICS Classification Rigidity through Stock Correlations

Tyler Christopher Lin, Brandon Zento Li, Houssam Kadri, Frank Huang

April 2, 2026

1. Problem Statement

The prediction of the stock market remains one of the most persistent challenges in quantitative finance. Traditional financial analysis relies heavily on the Global Industry Classification Standard (GICS), a hierarchical framework classifying over 43,000 publicly traded companies into 11 broad sectors such as "Technology" and "Energy."

The GICS, however, rests on the assumption that companies within the same category exhibit similar price behaviors, while companies across categories behave differently. In practice, stock market value is driven by geopolitical and socioeconomic events, making consistently reliable predictions exceptionally difficult. A single major economic event can simultaneously affect multiple, seemingly unrelated industries through higher-order dependencies.

Our question: To what extent do traditional GICS sectors remain distinct to one another, and can graph-based detection identify 'pivot stocks' that connect supposedly unrelated industries?

Dataset(s) Used

This project's dataset is a subset of Yahoo's financial data¹. In particular, we use the S&P 100 companies when building the correlation graph. When the ticker data is downloaded, it returns in the form of a `pandas.DataFrame`. For more than one ticker, the columns are arranged as a multi-level index (price fields and ticker symbols); we flatten this in `download_tickers` in `data_reader.py` using `DataFrame.stack()` and `DataFrame.reset_index()` so that each row is one date and one ticker. The flattened table includes `Date`, `Ticker`, and OHLCV columns, where OHLCV stands for the usual daily fields: **O**pen (first trade price), **H**igh, **L**ow, **C**lose (last trade price), and **V**olume (shares traded). Table 1 shows the shape of that row-wise table with illustrative values; our program only reads `Date`, `Ticker`, and `Close` when we call `log_return_list()` in `compute.py` to build the log-return series. Under `yfinance`'s default `auto_adjust=True`, the `Close` column is adjusted for splits and dividends in line with the library's usual behavior. GICS sectors are not columns in that price table. We call `get_sectors_for_tickers()` separately, which uses `Ticker.info.get('sector', 'Unknown')` for each ticker, and we attach those sector strings to nodes in the `CorrelationGraph`.

Table 1: Illustrative rows of the flattened price `DataFrame` after `stack / reset_index` (not actual downloaded output).

Date	Ticker	Open	High	Low	Close	Volume
2025-01-02	AAPL	184.22	186.00	183.89	185.64	64220000
2025-01-02	MSFT	370.10	373.00	369.50	372.50	22450000

Computational Overview

As outlined in Section 1, we retrieve stock price data from the Yahoo Finance Python API, `yfinance`. With each ticker represented as a node in a graph, edges are formed based on the correlation between two nodes' price trends. If the strength of the correlation exceeds a user-specified threshold, an edge is formed between them. This forms an undirected weighted graph, which is central in analyzing price behaviors across sectors, such as whether an economic event affecting one sector propagates to another.

The overall computation pipeline accepts a user-specified list of thresholds, resulting in a graph being constructed for each threshold. However, the underlying price data is downloaded only once.

To construct a graph, the historical price data of the user-specified set of tickers is first downloaded with two additional user-specified parameters: either a start and end date or a time period, and the sampling frequency. Once downloaded, the resulting `pandas.DataFrame` is then restructured so that the `DataFrame` can be indexed by ticker and date. We then call `get_sectors_for_tickers()` with the ticker set to retrieve the sector of each company, and calculate the log returns, the natural logarithm of the ratio of consecutive stock prices, by calling `log_return_list()` with our `DataFrame`. Doing so produces one dictionary with the ticker names and corresponding GICS sectors, and another dictionary which pairs tickers to their respective list of log returns. We then add each node, which has attributes `ticker`, `sector`, and `neighbours`, into a `CorrelationGraph` object, forming the underlying graph structure. Edges are added by computing the Pearson Correlation Coefficient between each pair of nodes using their lists of log returns. The value is then compared to a user-specified threshold. An edge is added between two nodes if the absolute value of the Pearson coefficient is greater than the threshold to identify sufficiently strong positive and negative correlations.

The module used for data retrieval is `yfinance-cache`, an unofficial caching wrapper of `yfinance` designed to prevent rate-limiting by the Yahoo Finance API by caching previously retrieved data locally. Data is retrieved by calling the `yfinance_cache.download()` function. A `yfinance_cache.Ticker` object, instantiated with a ticker symbol, contains all available information for that stock. Calling `Ticker.info.get('sector', 'Unknown')` returns the sector of each ticker.

The module used for data storage and manipulation is `pandas`, a library providing high-performance data structures and tools to manipulate tabular data. Data retrieved from `yfinance_cache.download()` is returned as a `pandas.DataFrame` object. The `DataFrame` is then flattened from a multi-level column format to a row-wise format indexed by date and ticker by calling `DataFrame.stack()` and `DataFrame.reset_index()`. This format allows direct access to the close prices per ticker and date from which log returns are calculated.

We also implement the crash simulation and pivot-candidate ideas from our proposal inside `CorrelationGraph`. The function `bfs_crash_simulation(start_ticker)` runs a breadth-first search along correlation edges from a user-selected “crashing” stock, recording nodes layer by layer up to a maximum depth given by `BFS_DEPTH` in `constants.py`. The helper `_reachable_from_crash` unions those layers into one set of tickers, which we treat as the reach of the simulated shock. The function `get_pivot_candidates(start_ticker)` then looks at each ticker in that set: it counts how many distinct GICS sectors appear among that ticker and its graph neighbours that also lie in the set, and it keeps tickers where that count is greater than one. The return value is a list of `(ticker, sector_diversity)` pairs sorted so that the largest diversity appears first. This is a graph-based heuristic for stocks that bridge sectors under the simulation; it is not a ready-made list of pivot stocks taken from outside our program.

Visualization Overview

Our visualization is primarily created through the `plotly.graph_objects`, with the positioning of graph nodes controlled by `networkx`.

Our visualization used many aspects such as color and node size to improve viewer clarity. The edge traces are built such that positive correlations create green edges, while red edges indicate that the edge represents a negative correlation. The node sizes are normalized such that a node with a higher degree would appear relatively larger in size than another node with a lower degree. The nodes themselves are generally colored by sector. Additionally, when searching for pivot candidates from a start node, the colors change so that the start ticker and identified candidates are highlighted in different colours, while the rest of the nodes are greyed out.

Node and edge traces are built into `go.Scatter` objects, which are then built into a `go.Figure` object to be sent to the Plotly `dash` app. We used the `dash` library to build a Flask app around our `go.Figure`, since `dash` includes built-in core components to make inputs. This improves the user interface and allows for easier user control of our interactive graph. Our interface includes a slider to control the threshold, a radio menu to control the view options, and a drop-down search to search for pivot candidates from a start ticker. These inputs interface with the elements they update through functions that have the `@callback decorator`. These decorators mean that the function runs whenever the input is used.

To reduce the time it takes to change between different figures for different thresholds or view options, each combination of threshold and view option is precomputed. These figures are computed at the beginning of the pipeline and are stored in `figures_by_threshold`. This is a nested dictionary with the outer dictionary mapping

the three view options of 'Standard', 'Sector', and 'Community' to an inner dictionary that maps the threshold to the figure. This allows each figure to be quickly accessed. Thus, each time the threshold slider or view option is changed, we can update the figure by returning the value from our dictionary rather than going through the process of creating each figure every time. However, this approach is not feasible when we look for pivot candidates by starter ticker, as there are a large number of possible combinations. Thus, we build the traces for the figure each time we search for pivot candidates.

We split our visualization into 3 view options: **Standard, Sector, and Community**.

Standard view uses `NetworkX.spring_layout`, which "[treats] edges as springs holding nodes close, while treating nodes as repelling objects"⁹. This visually groups stocks into clusters where nodes with edge connections to one another would be closer together, and nodes that did not have connections would be repelled outwards to the edges. However, this did not create a clear viewing experience because most of the nodes clustered into 1 large group with only a few small groups and standalone nodes on the outside.

Sector view groups the nodes into clusters based on their GICS sector through a clever layout trick taken from the NetworkX documentation.¹⁰ We create a `nx.cycle_graph` with length equal to the number of sectors. This creates a circle-like graph in which all the nodes are contained in a singular cycle. The positions of these nodes are then used as the centers for spring layouts of subgraphs representing each sector. Thus, all nodes of the same sector are grouped in close proximity.

Community view uses `nx.greedy_modularity_communities` to group the stocks into graph communities, where the number of communities matches the number of GICS sectors in the graph. Communities are often defined as groups of vertices where the density in that group is higher than the average density in the graph.¹¹ Community view takes the same trick from Sector view, grouping the stocks proximally into the clusters that the NetworkX algorithm finds. Essentially, if you asked the algorithm to group the full stock dataset into 11 sectors instead of the 11 GIC sectors present, this view shows how it would form groupings.

At the bottom of our page, we also have panels showing useful statistics regarding the graph density and the average absolute Pearson coefficient of the sector subgraphs. Under the headers for these panels, we show the values for the full graph. Further underneath, we have the values for the sectors ranked from greatest to least, so that the viewer can compare the density and correlation metrics between sectors to judge relative accuracy in the groupings. Note that density updates with the threshold changes, but the average absolute Pearson coefficient of the sectors does not because it can only be done on a complete graph.

Running the Program

The provided file `requirements.txt` provides all the required modules to run the program. Note that when installing `yfinance-cache`, an underscore must be used instead of a hyphen, i.e. `pip install yfinance_cache`. You can run `pip install -r requirements.txt` to install on the necessary packages.

Because we are retrieving data through the Yahoo Finance API, this project does not rely on a static dataset, as mentioned previously. However, the project uses the S&P 100 tickers (as of 2024-2025) and their price data from March 2nd, 2026 to March 31st, 2026 as the project's dataset, provided in `py-yfinance-cache.zip`. The data must be extracted into `C:\Users\<Username>\AppData\Local\yfinance_cache` for the data to load in the program. When `main.py` is run, it takes the tickers from `constants.py` and retrieves the corresponding historical price data directly from the Yahoo Finance API. If the data already exists in the zip file, then it will be retrieved from there, significantly decreasing the retrieval time.

In the unlikely chance that the download fails, in the function call on line 36 of `main.py`, please replace the arguments with `tickers`, `thresholds`, `date_range=('2026-03-02', '2026-04-01')`, `interval='1d'` to retrieve exclusively the data included in the zip file.

When `main.py` is run and you are not using data from the provided zip file, you may notice that the result will return slowly, which is due to the nature of `yfinance_cache.download()`. With a larger sample, it will take some time to download each ticker's data before creating the graph.

The program visualization requires self-hosting. When the `main.py` is run, please follow the hyperlink that shows up after some time. This will lead to the opening of the graph visualization.

The following shows something similar to the initial appearance of the graph. Hovering the mouse over a node shows the node's ticker, GICS sector, and the number of neighbors. The larger a node, the more neighbors it has. In this mode, nodes are clustered by the number of correlations with other nodes. On the right hand side of the screen is a toggleable legend which hides the corresponding symbol on the graph upon clicking, as well as a choice of node grouping and a ranking of the nodes with the most connections. The bottom contains a slider to set the threshold

level, and scrolling down will reveal additional information about the graph such as graph density and more detailed Pearson coefficient data for each sector.

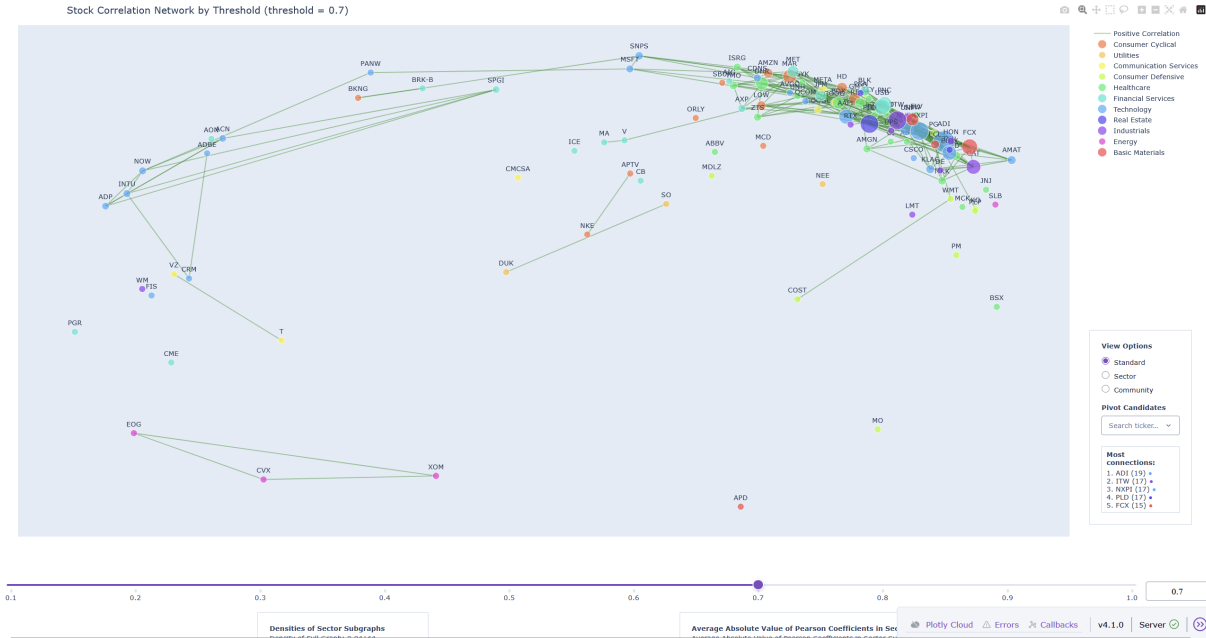


Figure 1: An image of what the graph could look like upon opening the site.

As mentioned previously, using the slider at the bottom will show only the edges whose absolute values exceed the threshold. The corresponding information changes along with the graph.

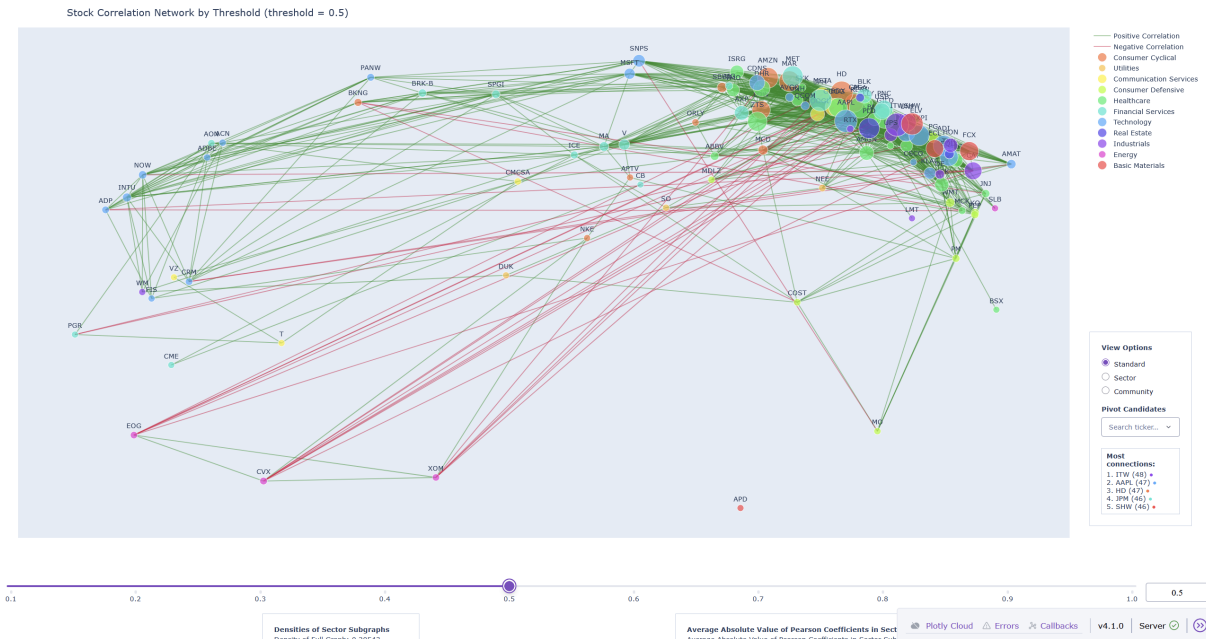


Figure 2: Stock Network Correlation Graph - 0.5 Threshold

On the right below the legend you will find **View Options** and **Pivot Candidates**.

First, **View Options** allows different graph orientations, as mentioned in the Visualization Overview, may aid in analysis. Click on the button to the corresponding viewing option to toggle it.

Then, **Pivot Candidates** allows you to select any node in the graph as a target stock, highlighted in yellow. Typing or selecting a valid node name from the search bar will show that target's pivot candidates, highlighted in purple.

Furthermore, once a target stock is selected, you can select any of its candidates to become your new target by clicking on any of the candidates listed below the search bar. The pivot candidates menu can always be exited by clicking the 'X' icon in the menu's search bar.

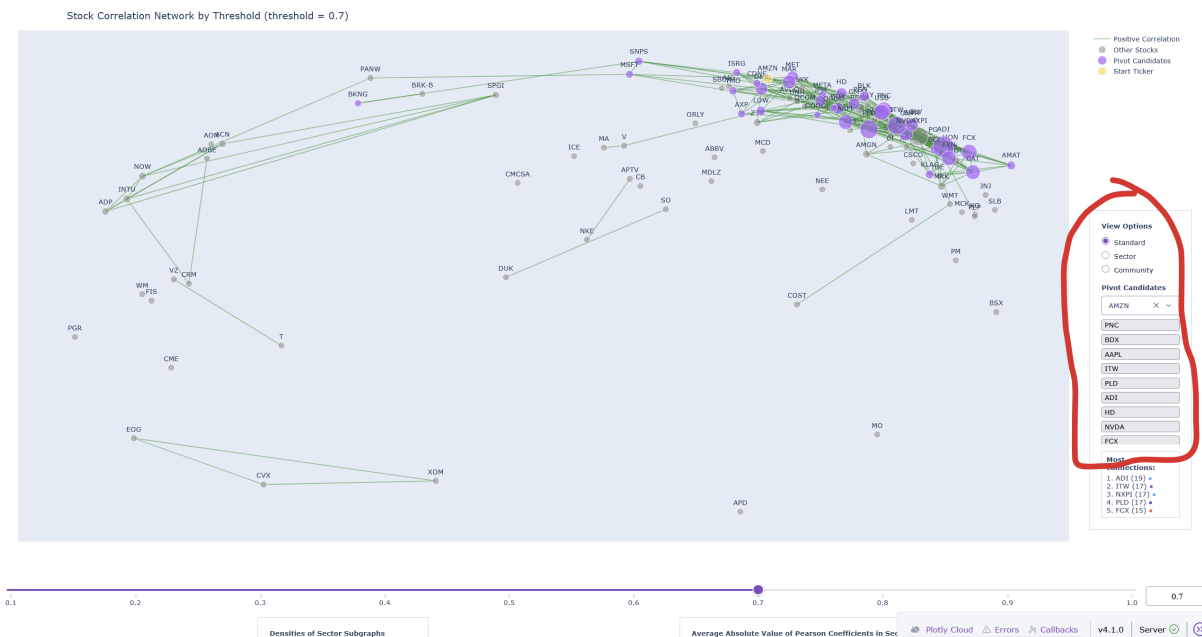


Figure 3: Correlation Graph with AMZN as its target stock.

Finally, on the top right corner of the site, there is the Plotly Modebar:

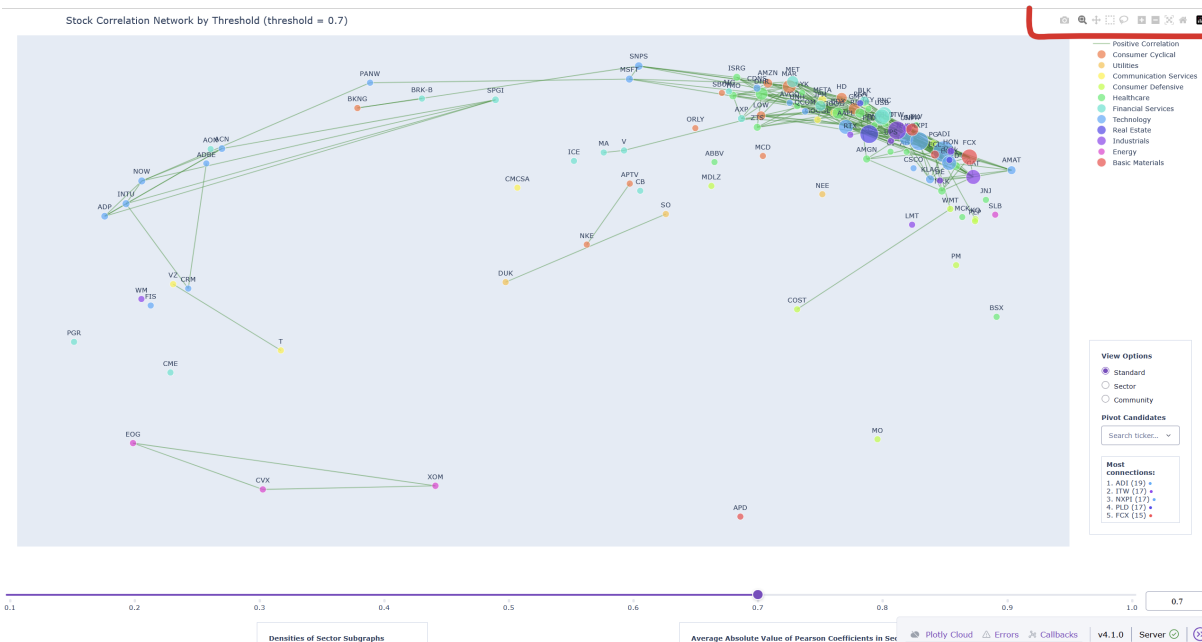


Figure 4: The Plotly Modebar, outlined in red.

With it, you can zoom in and out and pan around the graph, along with a few other actions.

Changes from Proposal

A few changes were made from our original proposal:

- Generalized the problem statement. Initially, the project aimed to analyze sector distinctness during periods of high volatility. Upon further discussion, the group concluded that, ideally, the graph would provide meaningful results during any economic period.
- Implementing a mode that switches clustering in the graph from the number of edges to grouping the nodes by sector. This was inspired by our TA and provides a different perspective in the analysis.
- A slider in the visualization that changes the correlation threshold, influencing the results. This change was also inspired by our TA and made narrowing down very strong correlations very easy without running the program multiple times.
- Instead of using our Breadth-First Search algorithm to simulate a stock crash, the project used it instead to find stocks that have a strong correlation across multiple GICS sectors.
- The additional data upon hovering over each node consists of only the ticker name, sector, and number of neighbours. Furthermore, a leaderboard was added to showcase the five tickers with the most neighbours. Our group decided that data such as the log return was irrelevant to the representation when the graph is focused solely on correlations, so we aimed to keep the data informative yet presentable.
- Changing the size of a node based on the number of correlations. This way, the graph felt more dynamic and it was easier to pinpoint "important" nodes within the network.

Discussion

This project set out to determine whether or not GICS-defined sectors accurately reflect the behavior of stocks measured by price correlation. To investigate, a correlation graph was constructed with the S&P-100 as the ticker set, with one month of historical prices retrieved at daily intervals.

At low correlation thresholds (0.5-0.6), results showed many edges within and across sectors, suggesting a relatively connected stock market, and that a node being a part of a sector is not the sole factor that determines its price behaviour. In other words, certain stocks across sectors share a strong enough price behaviour correlation that is not reflected by the standards of GICS sectors.

As the threshold increases (0.7 - 0.8), the network becomes much sparser and more interpretable. Many weak cross-sector correlations disappeared, while stronger correlations remained intact; the same pattern observed at lower thresholds, however, remained. Some stocks maintained strong connections across sectors as seen through the sector-organised layout, while others fragmented or kept only a few strong links. This helps answer our research question: traditional sectors do retain some distinctness, but that distinctness becomes more visible only when correlations are filtered out. At lower thresholds, shared connections across the market blur lines between the GICS structure.

For example, at a threshold of 0.8, the full graph is still extremely sparse with very low density, while most stocks are isolated or only weakly connected, a select few stocks remain meaningfully connected. Furthermore, when using the "pivot-candidate" search on many of the remaining stocks, we notice that some of them still appear to influence the structure of the network beyond its own sector, even when there is no direct correlation edge remaining between them and certain other stocks.

We had implemented a threshold slider that goes from 0.1 to 1.0, but we discovered that only the thresholds mentioned above demonstrated significant results. Anything below a 0.5 threshold had too many edges and was therefore too dense to distinguish any pattern, while any threshold above 0.8 became too sparse, as there were too few edges to draw any meaningful conclusion. This made the 0.5 - 0.8 threshold range the optimal balance between visibility and interpretability.

Through pivot candidates, analyzing different thresholds, and graph node clustering by sector, we were able to identify strong correlations between sectors, implying that the GICS categorization is not absolute.

While the project was able to adequately answer our inquiry, it was not without limitations. Our first problem was with the nature of the Yahoo Finance API: downloading too much data too quickly from Yahoo Finance at once causes the site to rate-limit and block the IP address. This is what ultimately led us to use our project’s dataset, which is the S&P 100 (from 2024-2025), a mere 100 stocks out of the tens of thousands of stocks on Yahoo Finance. We also only retrieved data over one month, which is insufficient in large scale analysis. Despite this, the runtime of the program was still noticeably slow; with a larger dataset and a longer time period, the runtime would exponentially increase. Some of our functions, like the calculation for the average absolute value of Pearson coefficients in a graph run in $O(N^2)$, meaning that our program would only run at reasonable speeds for up to about 10,000 input stocks. Above all, our graph is only able to analyze stocks by their Pearson correlation coefficients, which represent linear correlations, effectively overshadowing non-linear correlations between stocks.

Overall, the results were able to answer our question: Many edges appear to be between sectors, which follows the principle behind GICS sectors. We firmly believe, however, that our project still has a long way to go. First, we could implement an algorithm that uses a more sophisticated measure than the Pearson coefficient in order to see a more complete picture of the correlation between certain stocks. Second, we could try to further optimize our code to be able to handle a much larger dataset over a longer time period with a reasonable runtime. While this will not solve the rate-limiting issue, an algorithm can be developed to download a certain number of ticker data over certain intervals.

References

1. Yahoo Finance. (2025). *Yahoo Finance — Stock Market Live, Quotes, Business & Finance News*. <https://finance.yahoo.com>
2. Aroussi, R. (2024). *yfinance: An Open-Source Tool*. <https://github.com/ranaroussi/yfinance>
3. The pandas development team (2024). *pandas-dev/pandas: Pandas Documentation*. <https://pandas.pydata.org/docs/>
4. Harris, C. R., et al. (2020). *Array programming with NumPy*. Nature. <https://numpy.org/doc/>
5. NetworkX Developers. (2024). *NetworkX Density Function Documentation*. <https://networkx.org/documentation/stable/reference/generated/networkx.classes.function.density.html>
6. Plotly Technologies Inc. (2024). *Interactive Graphing Library for Python*. <https://plotly.com/python/>
7. MSCI and S&P Dow Jones Indices. (2023). *GICS Sector Methodology Overview*.
8. *yfinance-cache* (Python package). Caching wrapper for yfinance. <https://pypi.org/project/yfinance-cache/>
9. “Spring layout — NetworkX 2.8.7 Documentation.” *Networkx.org*, networkx.org/documentation/stable/reference/generated/networkx.drawing.layout.spring_layout.html.
10. “Cluster Layout — NetworkX 3.6.1 Documentation.” *Networkx.org*, 2025, networkx.org/documentation/stable/auto_examples/drawing/plot_clusters.html.
11. Fortunato, Santo, and Claudio Castellano. “Community Structure in Graphs.” Springer EBooks, 1 Jan. 2012, pp. 490–512, https://doi.org/10.1007/978-1-4614-1800-9_33.
12. GeeksforGeeks. “How to Check If a Deque Is Empty in Python?” *GeeksforGeeks*, 28 Dec. 2022, www.geeksforgeeks.org/python/how-to-check-if-a-deque-is-empty-in-python/. Accessed 2 Apr. 2026.
13. Vrsalovic, Vinko. “Does Python Have a Ternary Conditional Operator?” *Stack Overflow*, 27 Dec. 2008, stackoverflow.com/questions/394809/does-python-have-a-ternary-conditional-operator. Accessed 2 Apr. 2026.

14. atharvakatre. “How to Set Box Select as the Default instead of Zoom on the Modebar?” *Plotly Community Forum*, 6 Mar. 2022, community.plotly.com/t/how-to-set-box-select-as-the-default-instead-of-zoom-on-the-modebar/61663/2. Accessed 2 Apr. 2026.
15. jctomas. “Set Intial Mode to Box Select.” *Plotly Community Forum*, 9 Apr. 2019, community.plotly.com/t/t/set-intial-mode-to-box-select/22073. Accessed 2 Apr. 2026.
16. BlackMath. “How to Change the Website Tab Name in Dash-Plotly Using Python.” *Stack Overflow*, 14 Oct. 2021, stackoverflow.com/questions/69570145/how-to-change-the-website-tab-name-in-dash-plotly-using-python.
17. tcbegley. “Dash / CSS: Make a Div Full Screen.” *Plotly Community Forum*, 13 Mar. 2021, community.plotly.com/t/dash-css-make-a-div-full-screen/50982/2. Accessed 2 Apr. 2026.
18. Fran. “How to Animate Dash Graphs — Dash-Resources.com.” *Dash-Resources.com*, 2 Feb. 2025, dash-resources.com/how-to-animate-dash-graphs/. Accessed 2 Apr. 2026.
19. Plotly. “Basic Callbacks — Dash for Python Documentation — Plotly.” *Dash.plotly.com*, dash.plotly.com/basic-callbacks.
20. Plotly. “Slider — Dash for Python Documentation — Plotly.” *Plotly.com*, 2024, dash.plotly.com/dash-core-components/slider.
21. Plotly. “RadioItems — Dash for Python Documentation — Plotly.” *Dash.plotly.com*, dash.plotly.com/dash-core-components/radioitems.